Projeto Final — Aprendizado de Máquina I

Parte II: Pipeline de Classificação

Relatório de Projeto

Grupo *Machine Learning Name in progress*

Ryan Kevin Mateus Bandeira Micaele Brandão Vinicius Flesch Lucas Vieira
Mateus Almeida Marcelo Lopes Gabriel Martinez

Aprendizado de Máquina I (2026.2)
Prof. Rodrigo Ribeiro

impatech

Resumo

Neste relatório, aplicamos a biblioteca de aprendizado de máquina desenvolvida pelo nosso grupo (**Machine Learning Name in progress**) para prever o gênero musical de faixas do conjunto de dados *30000 Spotify Songs* a partir de suas características de áudio. Comparamos cinco modelos (Baseline, LDA, QDA, Regressão Logística e KNN), usando validação cruzada em $k = 5$ partições para escolher os melhores hiperparâmetros e separando um conjunto de teste para a avaliação final. O modelo com melhor desempenho foi o KNN ($k = 35$), que atingiu F1-macro de 0,5450 na validação cruzada e 0,5318 no conjunto de teste (com acurácia de **0,5397**).

1 Introdução

1.1 Problema e motivação

O objetivo deste projeto é classificar músicas em seis gêneros principais (`edm`, `latin`, `pop`, `r&b`, `rap` e `rock`), representados pela coluna `playlist_genre`, usando apenas variáveis acústicas e informações temporais das faixas. Na prática, esse tipo de classificação automática é muito útil para sistemas de recomendação, organização de grandes catálogos em plataformas de *streaming* e criação automática de playlists. Além disso, é um bom caso de teste para a nossa biblioteca (`mlnip-lib`), já que gêneros musicais não têm divisões exatas e várias músicas misturam elementos de estilos diferentes.

1.2 Conjunto de dados

- **Fonte:** 30000 Spotify Songs (Kaggle).
- **Licença:** Domínio Público.
- **Tamanho original:** 32 833 linhas e 23 colunas.
- **Atributos usados:** popularidade (`track_popularity`), *danceability*, *energy*, *loudness*, *speechiness*, *acousticness*, *instrumentalness*, *liveness*, *valence*, *tempo*, duração (`duration_ms`), ano de lançamento (`release_year`), modo (`mode`, maior/menor) e tom (*key*, codificado com *one-hot*), somando 24 variáveis após o pré-processamento.

1.3 Métrica principal

Escolhemos o **F1-macro** como métrica principal (média simples do F1-score entre os seis gêneros). Como ele dá o mesmo peso para todas as classes, evita que o resultado seja distorcido pelo leve desbalanceamento que surgiu após a limpeza dos dados. Também calculamos acurácia, precisão-macro, recall-macro e especificidade-macro para complementar a análise.

2 Análise exploratória

Durante a exploração dos dados, identificamos alguns pontos importantes que guiaram a limpeza:

- **Músicas repetidas:** existem 28 356 `track_id` únicos em 32 833 linhas (4 477 repetições). Como uma mesma música pode aparecer em várias playlists, encontramos 1 687 faixas rotuladas com

mais de um gênero. Se mantivéssemos essas linhas, teríamos rótulos contraditórios e risco de a mesma música cair no treino e no teste ao mesmo tempo.

- **Vazamento de dados (*data leakage*):** a coluna `playlist_subgenre` entrega o gênero principal em 100% dos casos (cada um dos 24 subgêneros pertence a um único gênero), e `playlist_name`/`playlist_` fazem o mesmo em cerca de 99%. Por isso, descartamos essas variáveis.
- **Informação de álbum:** 92% dos álbuns têm músicas de um único gênero, então também removemos os nomes e IDs de álbuns.
- **Variáveis que mais separam as classes** (η^2): `release_year` (0,30), `speechiness` (0,20), `danceability` (0,19) e `energy` (0,14). Já `liveness`, `mode` e `key`, sozinhas, quase não ajudam a separar os gêneros.
- **Correlação entre variáveis:** as maiores correlações de Pearson são entre `energy` e `loudness` (0,68) e entre `energy` e `acousticness` ($-0{,}54$). Nenhum par passa de $|r| = 0{,}70$.
- **PCA:** precisamos de 10 das 12 variáveis numéricas para explicar 90% da variância, o que mostra que há pouca redundância linear nos dados.
- **Proporção das classes:** o desbalanceamento após a limpeza é pequeno, com razão de 1,25 entre a maior classe (`rap`, 4 993 faixas) e a menor (`latin`, 3 980 faixas).

3 Metodologia

3.1 Limpeza dos dados

1. Removemos as 1 687 músicas que apareciam com mais de um gênero.
2. Removemos as duplicatas de `track_id`, mantendo apenas um registro por música.
3. Removemos uma faixa que estava com `tempo` $= 0$ (erro na extração de áudio).

Com isso, a base passou de 32 833 linhas para **26 668 faixas únicas** (6 165 linhas removidas).

3.2 Divisão treino/teste

Fizemos a divisão estratificada por gênero usando semente fixa (`SEED = 67`):

- **Treino:** 21 335 faixas (80%), usadas nos testes iniciais e na validação cruzada;
- **Teste:** 5 333 faixas (20%), guardadas apenas para a avaliação final do melhor modelo.

3.3 Pré-processamento

- Descarte de IDs (`track_id`, `track_album_id`, `playlist_id`), colunas de texto (`track_name`, `track_artist`, `track_album_name`, `playlist_name`) e da coluna `playlist_subgenre`.
- Extração apenas do ano numérico (`release_year`) a partir de `track_album_release_date`.

- Aplicação de *one-hot encoding* na variável `key` (usando a função `one_hot` da nossa biblioteca), descartando `key_0` para evitar colinearidade com o intercepto.
- Padronização (z-score) das 12 variáveis numéricas contínuas com a função `standardization`.

Para não vazar informação para a validação ou para o teste, as médias, desvios padrão e categorias foram calculados apenas nos dados de treino de cada etapa (incluindo dentro de cada partição da validação cruzada).

3.4 Modelos avaliados

Usamos os modelos e transformações implementados na biblioteca `mlnip-lib`:

Tabela 1: Modelos testados e grade de hiperparâmetros na validação cruzada.

Modelo	**Implementação**	**Hiperparâmetros testados**
Baseline	`MajorityClass` (notebook)	—
LDA	`LDA`	`reg_param` $\in \{0{,}0;\ 0{,}1;\ 0{,}3;\ 0{,}5;\ 0{,}9\}$
QDA	`QDA`	`reg_param` $\in \{0{,}0;\ 0{,}1;\ 0{,}3;\ 0{,}5;\ 0{,}7;\ 0{,}9\}$
Regressão logística	`LogisticRegression` (um-contra-todos)	`learn_rate` $= 5{,}0$, `max_iter` $= 5000$
KNN	`KNN`	$k \in \{15,\ 35,\ 75\}$ (e testes rápidos em $\{5,\ 151\}$)

Nota sobre a regressão logística: como a classe `LogisticRegression` da biblioteca foi feita para problemas binários, criamos no próprio notebook o adaptador `OneVsRestClassifier`, que treina um modelo binário para cada gênero ($K = 6$) e escolhe a classe com maior probabilidade.

3.5 Protocolo de validação

- **Teste rápido (*holdout*):** separamos 80/20 dentro do próprio conjunto de treino (17 069 para treino e 4 266 para validação) para checar se tudo rodava corretamente e medir o tempo de cada modelo.
- **Escolha de hiperparâmetros:** rodamos validação cruzada em $k = 5$ partições (`k_fold_split`) nas 21 335 faixas de treino, refazendo a padronização dentro de cada *fold*.
- **Teste final:** retreinamos o melhor modelo da validação cruzada com todas as 21 335 faixas de treino e avaliamos uma única vez nas 5 333 faixas de teste.

4 Resultados

4.1 Teste rápido (*holdout*)

Tabela 2: Resultados na divisão interna 80/20 do conjunto de treino.

Modelo	**F1 treino**	**F1 val.**	**Acurácia val.**	**Treino (s)**	**Pred. (s)**
Baseline (`MajorityClass`)	0,053	0,053	0,187	0,028	0,000
LDA (`reg_param=0`)	0,491	0,502	0,508	0,035	0,003
QDA (`reg_param=0`)	0,501	0,482	0,497	0,039	0,012
Regressão logística (OvR)	0,482	0,499	0,519	2,755	0,001
KNN ($k = 15$)	0,575	0,541	0,548	0,000	115,950

4.2 Validação cruzada ($k = 5$)

A Tabela 3 resume o melhor resultado de cada modelo na validação cruzada, enquanto a Tabela 4 mostra todas as combinações de hiperparâmetros que testamos.

Tabela 3: Melhor configuração de cada modelo na validação cruzada em $k = 5$ partições (média $\pm$ desvio padrão).

Modelo	**Melhor hiperparâmetro**	**F1-macro (CV)**	**Acurácia (CV)**
Baseline	—	$0{,}0526 \pm 0{,}0015$	$0{,}1873 \pm 0{,}0061$
QDA	`reg_param` $= 0{,}0$	$0{,}4829 \pm 0{,}0140$	$0{,}4985 \pm 0{,}0133$
Regressão logística	`learn_rate` $= 5{,}0$, `max_iter` $= 5000$	$0{,}4943 \pm 0{,}0080$	$0{,}5130 \pm 0{,}0079$
LDA	`reg_param` $= 0{,}0$	$0{,}4994 \pm 0{,}0083$	$0{,}5046 \pm 0{,}0083$
KNN	**k = 35**	$\mathbf{0{,}5450 \pm 0{,}0051}$	$\mathbf{0{,}5532 \pm 0{,}0054}$

Tabela 4: Resultados completos da validação cruzada ($k = 5$).

Família	Configuração	F1-macro (média ± dp)	Acurácia (média ± dp)
Baseline	`MajorityClass`	0,0526 ± 0,0015	0,1873 ± 0,0061
LDA	`reg_param` = 0,0	0,4994 ± 0,0083	0,5046 ± 0,0083
LDA	`reg_param` = 0,1	0,4985 ± 0,0079	0,5039 ± 0,0078
LDA	`reg_param` = 0,3	0,4956 ± 0,0091	0,5012 ± 0,0088
LDA	`reg_param` = 0,5	0,4922 ± 0,0097	0,4983 ± 0,0094
LDA	`reg_param` = 0,9	0,4836 ± 0,0076	0,4906 ± 0,0072
QDA	`reg_param` = 0,0	0,4829 ± 0,0140	0,4985 ± 0,0133
QDA	`reg_param` = 0,1	0,4812 ± 0,0133	0,4973 ± 0,0127
QDA	`reg_param` = 0,3	0,4741 ± 0,0108	0,4899 ± 0,0101
QDA	`reg_param` = 0,5	0,4700 ± 0,0110	0,4859 ± 0,0101
QDA	`reg_param` = 0,7	0,4662 ± 0,0091	0,4821 ± 0,0081
QDA	`reg_param` = 0,9	0,4608 ± 0,0070	0,4767 ± 0,0060
Reg. Logística	`lr` = 5,0, `iter` = 5000	0,4943 ± 0,0080	0,5130 ± 0,0079
KNN	$k = 15$	0,5303 ± 0,0053	0,5384 ± 0,0058
KNN	**k = 35**	**0,5450 ± 0,0051**	**0,5532 ± 0,0054**
KNN	$k = 75$	0,5422 ± 0,0043	0,5508 ± 0,0048

4.3 Avaliação final no conjunto de teste

O **KNN com** $k = 35$ teve o melhor resultado na validação cruzada, tanto em F1-macro quanto em acurácia, além de variar bem pouco entre as partições. Por isso, ele foi o escolhido para o teste final. A Tabela 5 mostra o desempenho do modelo em cada gênero nas 5 333 músicas de teste.

Tabela 5: Desempenho por gênero no conjunto de teste ($n = 5\,333$) para o `KNN`($k = 35$).

Gênero	Precisão	Recall	Especificidade	F1	Suporte
`edm`	0,5640	0,7323	0,8733	0,6372	975
`latin`	0,4601	0,4347	0,9105	0,4470	796
`pop`	0,3584	0,3549	0,8846	0,3566	820
`r&b`	0,5118	0,4393	0,9160	0,4728	890
`rap`	0,5878	0,5796	0,9063	0,5837	999
`rock`	0,7387	0,6530	0,9560	0,6932	853
Média macro	**0,5368**	**0,5323**	**0,9078**	**0,5318**	5 333

Acurácia no teste: 0,5397 (53,97%). **Tempo de predição no teste:** 154,69 s (o tempo de treino é praticamente zero, já que o KNN apenas guarda os pontos).

5 Discussão

- **Efeito dos hiperparâmetros:** Tanto no **LDA** quanto no **QDA**, aumentar a regularização (`reg_param` de 0,0 a 0,9) só piorou o F1-macro (caindo de 0,4994 para 0,4836 no LDA, e de 0,4829 para 0,4608 no QDA). Isso faz sentido porque temos muitas linhas para poucas colunas

($n \approx 17\,068$ por partição contra $p = 24$ atributos), então as matrizes de covariância já são bem estimadas sem precisar de regularização. Aumentar esse parâmetro acaba ignorando correlações importantes, como entre `energy`, `loudness` e `acousticness`. Também notamos que o LDA foi melhor que o QDA (0,4994 contra 0,4829): estimar uma matriz de covariância para cada classe não trouxe ganho real, provavelmente porque as colunas binárias do *one-hot encoding* de `key` fogem bastante de uma distribuição normal.

No **KNN**, deu para ver bem o equilíbrio entre viés e variância: valores pequenos de k ($k = 5$ com F1 de 0,4983 e $k = 15$ com 0,5303) erram mais porque os gêneros se misturam bastante; o resultado melhora em $k = 35$ (F1 = 0,5450) e volta a cair aos poucos em $k = 75$ (0,5422) e $k = 151$ (0,5359), quando a vizinhança fica grande demais.

- **Validação × teste:** O `KNN` ($k = 35$) fez $0{,}5450 \pm 0{,}0051$ de F1-macro na validação cruzada e 0,5318 no teste (acurácia de 0,5532 na validação e 0,5397 no teste). A queda foi bem pequena (1,3 ponto percentual), mostrando que o modelo generalizou bem e que remover as músicas repetidas antes de dividir os dados evitou vazamento entre treino e teste.

- **Desempenho por gênero:** Os melhores resultados no teste foram em `rock` (F1 = 0,6932) e `edm` (F1 = 0,6372), seguidos por `rap` (F1 = 0,5837). São gêneros com características bem marcadas nos dados: o `rock` tem músicas mais antigas (`release_year` menor), menos dançantes e com alta energia; o `rap` se destaca pelo `speechiness` alto; e o `edm` junta alta energia, batida rápida (`tempo`) e maior `instrumentalness`.

  Por outro lado, `pop` teve o pior F1 (0,3566), seguido de `latin` (0,4470) e `r&b` (0,4728). Como vimos na análise exploratória, o `pop` fica bem no meio da distribuição na maioria das variáveis e acaba se confundindo com quase todos os outros estilos (tem músicas dançantes parecidas com `latin` e `edm`, faixas mais próximas de `r&b` e outras de `rock`).

- **Tempo de execução:** A diferença de tempo entre os modelos foi grande. O **LDA** treina em 0,035 s e faz as predições em 0,003 s (F1 de 0,4994), e a **Regressão Logística OvR** leva 2,76 s para treinar e 0,001 s para predizer (F1 de 0,4943). Já o **KNN ($k = 35$)** ganha cerca de 4,6 pontos percentuais em F1 (0,5450), mas demora 154,69 s para prever o conjunto de teste, porque precisa calcular a distância de cada música de teste para todas as 21 335 de treino. Ou seja, para ter a melhor qualidade offline o KNN vale a pena, mas se precisássemos de respostas rápidas em tempo real, o LDA ou a Regressão Logística seriam escolhas muito mais práticas.

6 Conclusão

Montamos todo o pipeline de classificação multiclasse usando apenas as funções de pré-processamento, divisão de dados, métricas e modelos que implementamos na biblioteca `mlnip-lib`. Todos os modelos ficaram bem acima do baseline (F1-macro = 0,0526). Os modelos lineares (LDA e Regressão Logística) chegaram perto de 0,50 de F1-macro, enquanto o **KNN ($k = 35$)** conseguiu lidar melhor com as fronteiras não lineares entre os gêneros e teve o melhor resultado geral, fechando com F1-macro de **0,5318** e acurácia de **53,97%** no teste mais de três vezes o acerto de um chute aleatório entre 6 classes.

- **Limitações do trabalho:**

  1. Os dados do Spotify trazem apenas médias gerais da música inteira (energia média, valência média etc.), o que perde mudanças importantes ao longo da faixa.
  2. O rótulo `playlist_genre` vem da playlist onde a música estava, e não do gênero da música em si por isso uma faixa pop pode aparecer em uma playlist de treino de academia ou de outro estilo.
  3. Nossa implementação do KNN calcula todas as distâncias no braço (força bruta), o que deixa a predição lenta quando a base de treino cresce.

Reprodutibilidade

- **Semente aleatória:** `SEED = 67` em todas as divisões (`train_test_split`, `k_fold_split` e inicializações).
- **Versão da biblioteca `mlnip-lib`:** 1.0.0.